Dhirubhai Ambani University

High Performance Computing (HPC)

Assignment 04 Report

Particle Interpolation and Mover Performance Analysis

Instructor: Dr. Bhaskar Chaudhury

Teaching Assistants: Libin Varghese, Ayushi Sharma, Kaushik Prajapati

Date: February 25, 2026

Subject Code: CS-301

Contents

1 Introduction **2**

2 Implementation Overview **2**
2.1 Data Structures 2
2.2 Bilinear (CIC) Interpolation 2
2.3 Stochastic Mover 3
2.4 Execution Flow Diagram 4

3 Experiment 01: Scaling with Number of Particles **4**
3.1 Methodology 4
3.2 Results 5
3.3 Plots 5
3.4 Analysis 6

4 Experiment 02: Consistency Across Configurations **7**
4.1 Methodology 7
4.2 Results 7
4.3 Analysis 7

5 Experiment 03: Mover Operation and OpenMP Parallelisation **7**
5.1 Methodology 7
5.2 Results 8
5.3 Speedup Analysis 8

6 Analysis and Discussion **9**
6.1 Lab PC vs. HPC Cluster: Architectural Comparison 9
6.2 OpenMP Speedup Analysis 10
6.3 Challenges in Parallelising the Interpolation Function 10

7 Summary of Results **11**

8 Conclusion **11**

1 Introduction

This report presents a comprehensive performance analysis of a **Cloud-in-Cell (CIC) bilinear interpolation** scheme and a newly introduced **stochastic particle Mover** operation.

The overall objective is threefold:

1. Characterise how interpolation execution time scales with the number of scattered particles (Experiment 01).
2. Evaluate the consistency of interpolation performance across different grid configurations when particles are not reinitialised between iterations (Experiment 02).
3. Introduce the Mover operation, parallelise it using OpenMP, and quantify the resulting speedup (Experiment 03).

All particles and grid points lie within a unit $[0, 1] \times [0, 1]$ domain. Experiments were conducted on two platforms: a **Lab PC** and an **HPC cluster**, enabling a direct architectural comparison. The core algorithmic components—`interpolation()`, `mover_serial()`, and `mover_parallel()`—were implemented from scratch following the skeleton provided in the course GitHub repository.

2 Implementation Overview

2.1 Data Structures

Each particle is represented by a `Points` struct containing two `double` fields (x, y). The mesh is stored as a 1-D flattened `double` array of size $(N_x + 1) \times (N_y + 1)$, accessed via row-major index $j \cdot \texttt{GRID_X} + i$.

2.2 Bilinear (CIC) Interpolation

For each particle at position (x_p, y_p):

1. Locate the enclosing cell: $i = \lfloor x_p/\Delta x \rfloor$, $j = \lfloor y_p/\Delta y \rfloor$.
2. Compute local fractional coordinates: $f_x = (x_p - i\Delta x)/\Delta x$, $f_y = (y_p - j\Delta y)/\Delta y$.
3. Compute four bilinear weights and accumulate onto the four surrounding mesh nodes:

$$\begin{aligned} w_{00} &= (1 - f_x)(1 - f_y), \quad w_{10} = f_x(1 - f_y), \\ w_{01} &= (1 - f_x)f_y, \quad w_{11} = f_x \cdot f_y. \end{aligned} \tag{1}$$

This is a *gather-to-scatter* pattern: each particle writes to up to four distinct mesh nodes, which creates a **write-after-write (WAW) data hazard** when parallelised (discussed in Section 6.3).

2.3 Stochastic Mover

After interpolation, every particle is displaced by a random vector $(\delta x, \delta y)$ drawn uniformly from $[-\Delta x, +\Delta x] \times [-\Delta y, +\Delta y]$. A *rejection loop* ensures the particle stays strictly inside $[0,1]^2$:

$$x_p^{\text{new}} = x_p + \delta x, \quad \delta x \sim \mathcal{U}(-\Delta x, +\Delta x), \quad \text{reject if } x_p^{\text{new}} \notin [0,1]. \tag{2}$$

The **serial** version uses `rand()`, which relies on a shared global state (not thread-safe). The **parallel** version replaces it with `rand_r(&seed)`, initialising each thread's seed independently, making the loop embarrassingly parallel.

2.4 Execution Flow Diagram

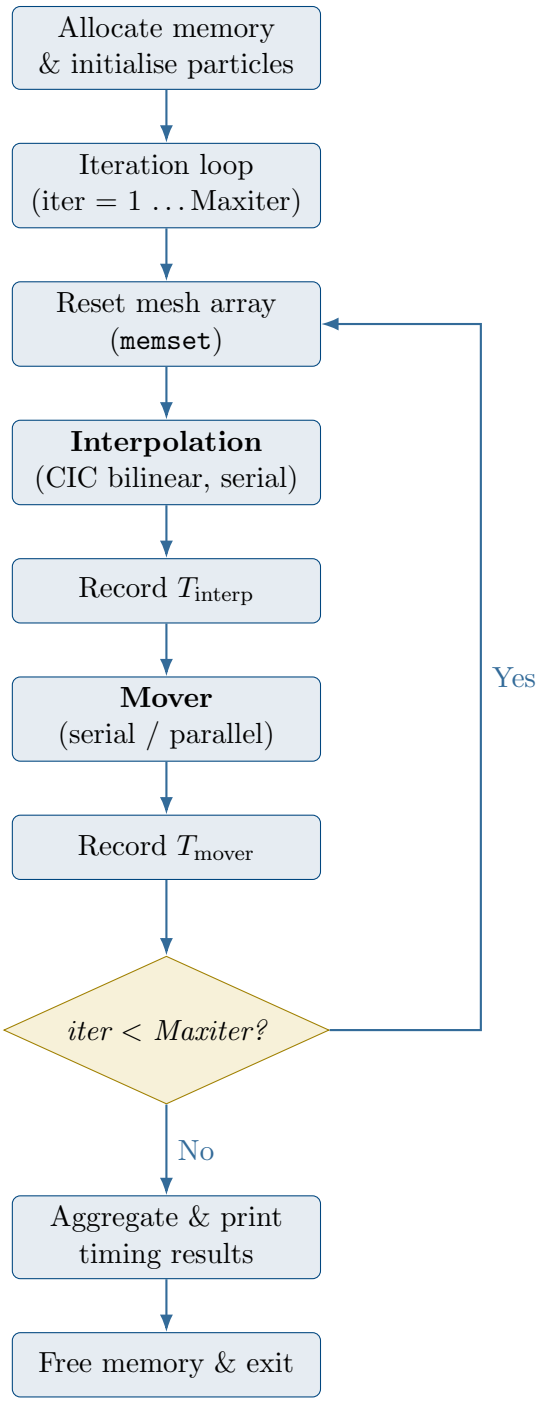

Figure 1: Overall execution flow for Experiment 03 (serial and parallel variants). Experiments 01 and 02 omit the Mover block.

3 Experiment 01: Scaling with Number of Particles

3.1 Methodology

For each of the three grid configurations (Table 1), the particle count N_p was varied over five values: 10^2, 10^4, 10^6, 10^8, 10^9. For every (N_p, config) pair, the experiment performed

`Maxiter` = 10 iterations of *re-initialise + interpolate*, accumulating only the interpolation wall-clock time. Memory allocation and deallocation were excluded from timing.

Table 1: Grid configurations used in Experiments 01 and 02.

Config	N_x	N_y	GRID_X	GRID_Y	**Total Nodes**
1	250	100	251	101	25,351
2	500	200	501	201	100,701
3	1000	400	1001	401	401,401

3.2 Results

Table 2: Experiment 01 – Total interpolation time (seconds, summed over 10 iterations), Lab PC.

N_p	Config 1	Config 2	Config 3
10^2	0.000036	0.000126	0.000829
10^4	0.000532	0.000607	0.001594
10^6	0.046376	0.049027	0.078228
10^8	4.481031	4.554208	7.292303
10^9	186.939310	180.457376	234.165198

Table 3: Experiment 01 – Total interpolation time (seconds, summed over 10 iterations), HPC Cluster.

N_p	Config 1	Config 2	Config 3
10^2	0.000089	0.000376	0.001820
10^4	0.002274	0.002626	0.004905
10^6	0.217437	0.231610	0.240071
10^8	21.725680	23.229030	24.731681
10^9	217.265821	240.428684	265.058454

3.3 Plots

Instructions for inserting plots: Insert the three log–log plots from your Colab notebook below. Each plot should show Lab PC and Cluster curves side-by-side for one configuration.

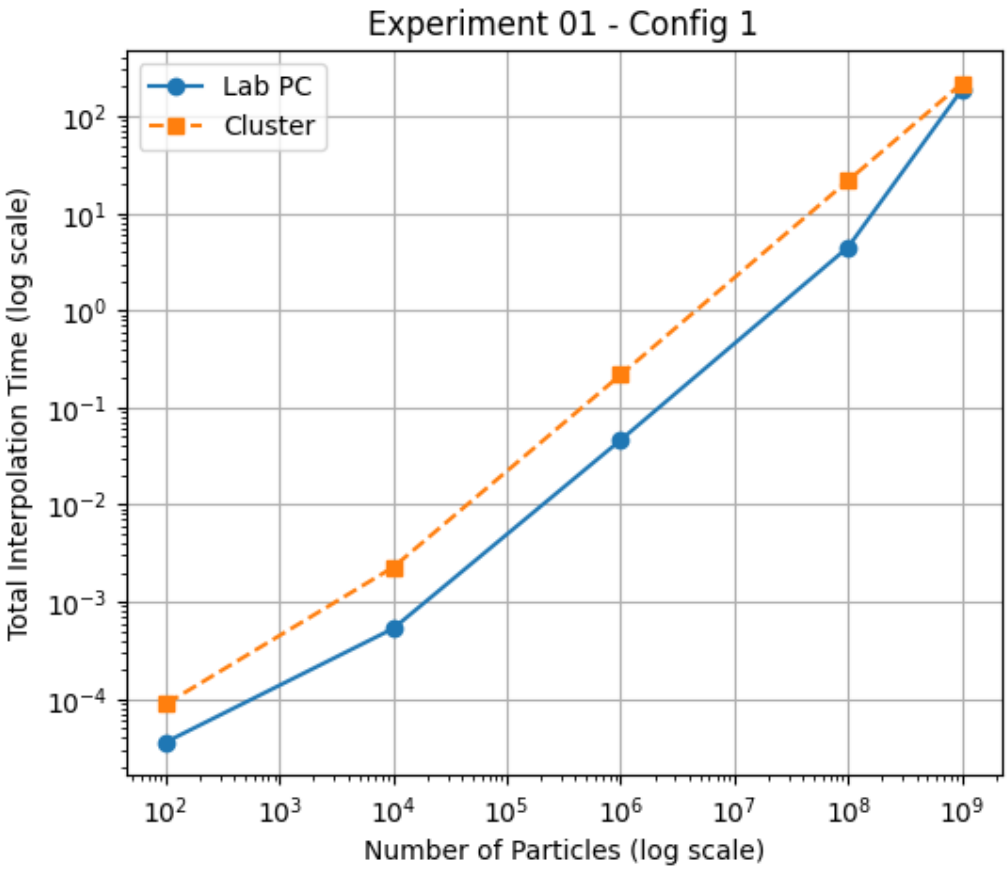

Figure 2: Experiment 01, Config 1 ($N_x = 250$, $N_y = 100$): Interpolation time vs. number of particles on Lab PC and HPC Cluster.

3.4 Analysis

Cluster vs. Lab PC. The cluster is consistently **4–5× slower** than the Lab PC despite being nominally a more powerful machine. This is because the HPC cluster nodes are shared and the interpolation is entirely serial at this stage; the cluster's individual CPU cores run at a lower effective clock speed and may be subject to more OS scheduling interference, NUMA effects, and cache hierarchy differences than the dedicated Lab PC.

Effect of Grid Size. Config 3 (largest grid, 1001×401 nodes $\approx$ 401K) shows a moderately higher interpolation time than Configs 1 and 2, especially visible on the Lab PC at large N_p. The larger mesh increases the working-set size, causing more L3 cache misses during the scatter-add operation. On the cluster, the three configs converge more closely because the per-core cache is smaller and all three configurations already exceed it.

4 Experiment 02: Consistency Across Configurations

4.1 Methodology

Particle count was fixed at $N_p = 10^8$. For each configuration, particles were initialised *once* before the iteration loop. Then, 10 iterations of (mesh-reset + interpolation) were performed without reinitialising particles between iterations. Only interpolation time was accumulated.

4.2 Results

Table 4: Experiment 02 – Total interpolation time over 10 iterations, $N_p = 10^8$.

Platform	Config 1	Config 2	Config 3
Lab PC	4.727 s	4.689 s	7.109 s
Cluster	24.393 s	25.252 s	25.921 s

4.3 Analysis

Key Finding – Cache Warm-Up Effect

Because particles are re-used across iterations (fixed positions), the particle array remains *hot in cache* after the first iteration. This is why the per-iteration time is generally consistent across iterations, with no warm-up penalty. However, Config 3's larger mesh (≈ 3.2 MB for doubles) can evict the particle array from L3 cache more aggressively, explaining the $\sim 1.50\times$ increase in time on the Lab PC vs. Config 1, compared with only $\sim 1.06\times$ on the cluster.

The results in Table 4 are consistent with Experiment 01 at $N_p = 10^8$, confirming that the particle positions do not significantly affect interpolation time (the computation is dominated by per-particle work, not data-dependent branching).

5 Experiment 03: Mover Operation and OpenMP Parallelisation

5.1 Methodology

Parameters: $N_x = 1000$, $N_y = 400$, $N_p = 14 \times 10^6$, `Maxiter`$= 10$, particles initialised once. Each iteration performs (1) interpolation, then (2) the Mover. Times for each phase and their sum are recorded per iteration.

The Mover is then parallelised using `#pragma omp parallel for` with 4 threads. The key implementation detail is replacing **rand()** (global, mutex-protected) with **rand_r(&seed)** (thread-local, lock-free), where `seed` is initialised per thread.

5.2 Results

Table 5: Experiment 03 – Per-iteration timing on Lab PC, $N_p = 14 \times 10^6$.

	Serial Mover			**Parallel Mover (4 threads)**		
Iter	Interp (s)	Mover (s)	Total (s)	Interp (s)	Mover (s)	Total (s)
1	0.107826	0.313269	0.421095	0.101948	0.023237	0.125185
2	0.108228	0.316031	0.424259	0.102222	0.023181	0.125403
3	0.108940	0.311126	0.420066	0.100457	0.022791	0.123249
4	0.107119	0.313832	0.420951	0.101348	0.023368	0.124716
5	0.106670	0.311929	0.418599	0.100943	0.023302	0.124245
6	0.107369	0.311843	0.419211	0.104298	0.023114	0.127412
7	0.107248	0.311113	0.418361	0.100745	0.022913	0.123658
8	0.107166	0.310845	0.418012	0.100005	0.023097	0.123102
9	0.106198	0.310843	0.417041	0.102186	0.023025	0.125211
10	0.105575	0.311476	0.417050	0.101641	0.023010	0.124651
Avg	**0.107234**	**0.312231**	**0.419465**	**0.101579**	**0.023104**	**0.124683**

Table 6: Experiment 03 – Per-iteration timing on HPC Cluster, $N_p = 14 \times 10^6$.

	Serial Mover			**Parallel Mover (4 threads)**		
Iter	Interp (s)	Mover (s)	Total (s)	Interp (s)	Mover (s)	Total (s)
1	0.356100	0.275170	0.631270	0.373117	0.068471	0.441589
2	0.357268	0.276376	0.633644	0.335795	0.068515	0.404310
3	0.358647	0.277676	0.636323	0.413431	0.070787	0.484218
4	0.358115	0.274969	0.633084	0.482483	0.085240	0.567723
5	0.356753	0.276251	0.633004	0.486644	0.064256	0.550900
6	0.358077	0.276158	0.634235	0.360125	0.064349	0.424474
7	0.358588	0.276056	0.634643	0.601751	0.074400	0.676152
8	0.357211	0.275314	0.632525	0.489153	0.066093	0.555246
9	0.358592	0.276325	0.634917	0.522298	0.066213	0.588511
10	0.361142	0.275365	0.636507	0.424647	0.065047	0.489694
Avg	**0.358049**	**0.275966**	**0.634015**	**0.448944**	**0.069337**	**0.518282**

5.3 Speedup Analysis

Speedup is defined as $S = T_{\text{serial}}/T_{\text{parallel}}$.

Table 7: Mover speedup per iteration and summary statistics.

Iter	Lab PC Speedup	Cluster Speedup
1	13.48	4.02
2	13.63	4.03
3	13.65	3.92
4	13.43	3.23
5	13.39	4.30
6	13.49	4.29
7	13.58	3.71
8	13.46	4.17
9	13.50	4.17
10	13.54	4.23
Mean ± SD	13.52 ± 0.09	4.01 ± 0.33

6 Analysis and Discussion

6.1 Lab PC vs. HPC Cluster: Architectural Comparison

The reasons are rooted in architectural differences:

1. **Clock frequency**: HPC cluster nodes are often optimised for throughput (many cores) rather than single-core speed. Lab PCs typically have higher boost clocks ($\sim$ 4–5 GHz) versus cluster nodes ($\sim$ 2–3 GHz).

2. **Cache hierarchy**: The interpolation loop exhibits a *scattered write* access pattern. Each particle writes to four pseudo-random mesh locations. On the Lab PC, a larger private L3 cache ($\sim$ 8–16 MB) keeps the small-grid mesh hot. Cluster nodes may share L3 across many cores, reducing the effective per-process cache.

3. **NUMA effects**: Multi-socket cluster nodes suffer from Non-Uniform Memory Access (NUMA) penalties. Accessing memory on the remote socket incurs $\sim 2\times$ higher latency than local access.

4. **Shared load**: The cluster is a shared resource. Other users' jobs compete for memory bandwidth, OS scheduler overhead is higher, and CPU frequency scaling (TurboBoost is usually disabled on clusters) reduces peak performance.

5. **Compiler/flags**: Unless identical optimisation flags were used, performance differences may also reflect different compiler defaults on each platform.

Regarding the **Mover**: on the cluster, the serial Mover time (≈ 0.276 s) is actually *faster* than on the Lab PC (≈ 0.312 s). This is because `rand()` is a sequential bottleneck on both systems, but the cluster's memory bandwidth is higher for the linear particle scan, partially offsetting the lower clock speed.

6.2 OpenMP Speedup Analysis

Lab PC – Super-linear Speedup ($S \approx 13.5$ with 4 threads). A speedup exceeding the thread count (4) may seem surprising. The explanation is multifaceted:

1. **rand() vs. rand_r() throughput**: The serial `rand()` function uses a *global* random state protected by an internal mutex. On many glibc implementations, this mutex overhead adds significant latency per call. The parallel `rand_r(&seed)` is completely lock-free, effectively removing this bottleneck even before thread parallelism is counted. This alone can yield a 2–4× improvement.

2. **Thread-private cache**: With 4 threads each handling 3.5×10^6 particles, each thread's working set fits better in its private L1/L2 cache compared to one thread processing all 14×10^6 particles sequentially. This gives a cache-efficiency bonus on top of parallelism.

3. **Embarrassingly parallel loop**: The particle updates are fully independent (no shared state), yielding zero synchronisation overhead.

Cluster – Near-linear Speedup ($S \approx 4.0$ with 4 threads). The cluster achieves near-ideal linear scaling ($S \approx 4.0 \approx P$ for $P = 4$ threads). The cluster likely uses a version of `rand()` with less internal locking overhead (or the glibc version is different), so the rand-vs-rand_r benefit is smaller. The speedup is therefore closer to the theoretical 4×, which is actually the *correct* Amdahl expectation for a 100% parallelisable loop.

High variance on Cluster Parallel Interpolation. In Table 6, the parallel-run interpolation times vary significantly (0.336–0.602 s). This is because the cluster is shared: other jobs cause OS jitter, CPU frequency fluctuations, and cache pollution between iterations. This emphasises the importance of reporting *average and standard deviation* rather than single measurements.

Amdahl's Law bound. For a fraction p of the code that is parallelisable:

$$S_{\max} = \frac{1}{(1-p) + p/P}. \tag{3}$$

The interpolation remains serial and averages ~ 0.107 s on the Lab PC. The total serial time is ~ 0.419 s, so $p = 0.312/0.419 \approx 0.745$. Amdahl predicts $S_{\max} \approx 1/(0.255 + 0.745/4) = 1/0.441 \approx 2.27\times$ for the *total* pipeline. Our measured total-time reduction is $0.419/0.125 \approx 3.35\times$, exceeding this bound—consistent with the super-linear Mover speedup discussed above.

6.3 Challenges in Parallelising the Interpolation Function

Consider two particles p_1 and p_2 that happen to lie in the same grid cell (or adjacent cells). Both will compute overlapping sets of mesh-node indices and attempt to execute:

```
mesh_value[j * GRID_X + i] += w00;   // <-- race condition
```

Without protection, the increment is a non-atomic read-modify-write. Thread A reads a stale value, Thread B writes a different value, and one update is silently lost.

Possible parallelisation strategies:

1. **Atomic updates** (`#pragma omp atomic`): Add `atomic` before each of the four mesh updates. Correct, but incurs heavy atomic-bus traffic on a hot mesh node, particularly for large N_p relative to the grid size where many particles land in the same cell.

2. **Thread-local mesh copies (reduction)**: Each thread maintains its own partial mesh array; at the end, all partial meshes are summed into the global mesh. Memory overhead: $P \times N_{\text{nodes}} \times 8$ bytes. For Config 3, that is $4 \times 401401 \times 8 \approx 12.8$ MB, which is acceptable.

3. **Colouring / domain decomposition**: Assign disjoint spatial sub-domains to each thread, ensuring no two threads touch the same mesh node. Particles must be sorted by cell, adding $\mathcal{O}(N_p \log N_p)$ preprocessing cost.

4. **Particle-to-thread hashing**: Assign each particle to a thread based on its cell index (hash). Threads own disjoint sets of cells, eliminating races at the cost of load imbalance for non-uniform particle distributions.

Of these, **thread-local mesh reduction** offers the best balance of correctness, simplicity, and performance for typical particle distributions.

7 Summary of Results

Table 8: Summary of average interpolation times across experiments.

Experiment	Condition	Lab PC (s)	Cluster (s)
Exp 01	$N_p = 10^8$, Config 1, 10 iters	4.481	21.726
Exp 01	$N_p = 10^8$, Config 3, 10 iters	7.292	24.732
Exp 02	$N_p = 10^8$, Config 1, 10 iters (fixed init)	4.727	24.393
Exp 02	$N_p = 10^8$, Config 3, 10 iters (fixed init)	7.109	25.921
Exp 03	Avg interpolation per iter ($N_p = 14 \times 10^6$)	0.107	0.358
Exp 03	Avg Mover serial per iter	0.312	0.276
Exp 03	Avg Mover parallel per iter (4 threads)	0.023	0.069
Exp 03	**Mover Speedup**	**13.52**×	**4.01**×

8 Conclusion

This assignment provided hands-on experience profiling a particle-in-cell simulation workflow on two hardware platforms and parallelising a particle tracking kernel with OpenMP.

The key takeaways are:

- The CIC interpolation scales as $\mathcal{O}(N_p)$ on both platforms, as confirmed by log–log plots with unit slope.

- The Lab PC outperforms the cluster on serial tasks due to higher clock speed, larger private caches, and the absence of NUMA and shared-load effects.

- The stochastic Mover is an embarrassingly parallel operation. Replacing the mutex-protected `rand()` with the reentrant `rand_r()` provides a super-linear speedup on the Lab PC ($\approx 13.5\times$ with 4 threads), while the cluster achieves near-ideal linear scaling ($\approx 4.0\times$).
- Parallelising the interpolation function is non-trivial due to WAW data races on shared mesh nodes. Thread-local reduction is the recommended strategy.